

Relatório Técnico

Assembly ARM64, Bits e Lookup Tables

Implementação completa dos 12 exercícios práticos do TP2, cobrindo lookup tables, mascaramento de bits, extração e inserção de campos, rotação manual e um pipeline completo de transformação. A validação foi feita com binário AArch64 executado em `qemu-aarch64`, conforme orientação do professor.

ALUNO Renato DISCIPLINA DR4 - Processamento Otimizado em Assembly ARM 64-bits ENTREGA TP2	AMBIENTE DE VALIDAÇÃO aarch64-linux-gnu-gcc + qemu-aarch64 ALVO PREVISTO Raspberry Pi Zero 2W / Linux ARM64 ARTEFATOS Código Assembly, suíte de testes, evidências em texto e relatório
--	--

1. Objetivo e escopo

O objetivo deste TP2 foi implementar doze exercícios em Assembly ARM64 com foco em técnicas de otimização escalar baseadas em lookup tables, mascaramento explícito de bits e composição manual de operações sobre palavras de 64 bits.

Todos os exercícios foram implementados dentro de um único módulo Assembly, compilados como binário AArch64 estático e validados localmente com `qemu-aarch64`. Essa estratégia preserva a compatibilidade com o ambiente Linux ARM64 esperado para o Raspberry Pi Zero 2W, mas dispensa a necessidade de usar o hardware físico durante a etapa de desenvolvimento.

Decisão de ambiente: o professor informou que a execução poderia ser validada em QEMU. Assim, a entrega foi estruturada para gerar um executável ELF 64-bit ARM aarch64, testado por cross-compilação e execução em emulação de usuário.

Pontos relevantes da entrega

- As 12 rotinas pedidas no enunciado foram implementadas em Assembly ARM64.
- As tabelas de consulta foram alocadas em `.rodata`.
- Foi criada uma suíte de testes automatizada para validar cada exercício.
- As evidências pedidas foram registradas como texto de terminal, sem imagens.
- O vídeo não foi produzido porque foi dispensado.

2. Organização do projeto

O diretório do TP foi reorganizado para separar implementação, testes, evidências e relatórios. Isso facilita tanto a reprodução da validação quanto a leitura pelo professor.

- Assembly ARM64
- Cross-compilação
- QEMU user mode
- Relatório HTML + PDF
- Evidência textual

Arquivo / pasta	Função
src/tp2_exercises.S	Implementa as 12 rotinas e define todas as lookup tables usadas nos exercícios.
include/tp2.h	Declara as interfaces das funções exportadas pelo módulo Assembly.
tests/test_tp2.c	Executa a validação automatizada com casos de teste e comparação de saída esperada.
Makefile	Coordena compilação, execução em QEMU e geração de evidências.
evidencias/qemu_test_output.txt	Log integral da suíte de testes em emulação AArch64.
evidencias/binario_info.txt	Metadados do executável gerado, confirmando o formato AArch64.

Fluxo de validação

1. Implementação

As rotinas foram escritas em Assembly ARM64, com convenção AAPCS64 e retorno pelos registradores `w0/x0`.

2. Cross-build

O projeto é compilado por `aarch64-linux-gnu-gcc` e ligado estaticamente.

3. Execução

O binário AArch64 é executado com `qemu-aarch64`, sem depender do Raspberry físico.

4. Evidências

O comando `make evidence` salva o texto completo da validação em arquivo.

```
make
make test
make evidence
```

3. Implementação dos exercícios

Os exercícios foram organizados em quatro grupos: lookup tables, operações por máscara, manipulação de campos e pipeline final. A tabela abaixo resume a solução técnica adotada em cada um deles.

Ex.	Implementação	Observação técnica
1	Lookup table de 256 bytes em .rodata, com leitura por ldrb.	Fórmula da tabela: $f[i] = (73*i + 41) \bmod 256$.
2	Lookup table de 256 halfwords, acessada por ldrrh [base, index, LSL #1].	Offset por elemento de 2 bytes; fórmula: $f[i] = (0x1200 + 257*i) \bmod 65536$.
3	Derivação explícita do índice com uxtb sobre a word de entrada.	Índice = 8 bits menos significativos; tabela: $f[i] = i \text{ XOR } 0xA5$.
4	Lookup table de words, lida por ldr [base, index, LSL #2].	Indexação correta por elementos de 4 bytes.
5	Rotina de limpeza com BIC.	Produce base & ~mask, preservando os demais bits.
6	Ativação de bits com ORR.	Implementa $base \mid mask$.
7	Alternância seletiva com EOR.	Somente os bits cobertos pela máscara são invertidos.
8	Teste por mascaramento com ANDS e CSET.	Retorna 1 para grupo não nulo e 0 caso contrário.
9	Extração genérica por shift right e máscara dinâmica.	Recebe valor, offset, largura e alinha o campo no LSB.
10	Inserção genérica por limpar-região + posicionar + ORR.	Preserva todos os bits fora do campo atualizado.
11	Rotate-left manual com dois shifts e ORR.	O deslocamento é normalizado com $N \& 63$.
12	Pipeline: extração de nibble, lookup em tabela de 16 bytes e reinserção em campo de 6 bits.	O resultado final também seta um bit de status em bit 40.

Detalhes do pipeline do exercício 12

Configuração adotada: o índice é extraído dos bits [15:12] da entrada. A lookup table tem 16 entradas e segue a fórmula $t[i] = (0x12 + 11*i) \bmod 256$. Em seguida, apenas os 6 bits menos significativos da saída da tabela são reposicionados no campo [25:20] do valor final, e o bit 40 é forçado para 1.

Lookup tables usadas

- Ex. 1: 256 entradas de 8 bits.
- Ex. 2: 256 entradas de 16 bits.
- Ex. 3: 256 entradas de 8 bits, com índice derivado.
- Ex. 4: 256 entradas de 32 bits.
- Ex. 12: 16 entradas de 8 bits para o pipeline final.

Instruções-chave

- LDRB/LDRH/LDR para acesso escalonado a tabelas.
- UXTB para derivação disciplinada de índice.
- BIC/ORR/EOR nas operações de máscara.
- ANDS + CSET para retorno de status em 0/1.
- LSL/LSR/ORR na rotação manual e no pipeline.

4. Validação e evidências

A validação foi automatizada em um harness AArch64. Cada exercício recebeu vários casos de teste, e o programa aborta com código de erro se alguma comparação falhar. O log completo da execução está salvo em `evidencias/qemu_test_output.txt`.

Confirmação do binário gerado

```
build/tp2_arm64_bits: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux),
statically linked, for GNU/Linux 3.7.0, with debug_info, not stripped
```

Resumo da suíte de testes

Grupo	Casos executados	Resultado
Exercícios 1 a 4	20 consultas em tabelas de byte, halfword e word	Todos aprovados
Exercícios 5 a 8	13 casos de mascaramento, toggle e teste de status	Todos aprovados
Exercícios 9 a 11	13 casos de extração, inserção e rotação manual	Todos aprovados
Exercício 12	4 casos de pipeline completo	Todos aprovados

Casos de teste do exercício 12

Entrada	Índice extraído	Valor da tabela	Saída final
0x123456789ABCDEF0	0xD	0xA1	0x123457789A1CDEF0
0x000000000000F123	0xF	0xB7	0x000001000370F123
0x13579BDF2468ACE0	0xA	0x80	0x13579BDF2408ACE0
0xFFFFFFFFFFFFFFFF	0xF	0xB7	0xFFFFFFFFF7FFFF

Trecho do output de execução

```
Validacao automatizada do TP2 - Assembly ARM64
Execucao cruzada: binario AArch64 rodando em qemu-aarch64

Exercicio 1 - Lookup table byte -> byte
[ OK ] ex1[0]: 0x29
[ OK ] ex1[255]: 0xe0

Exercicio 8 - Teste de bits por mascaramento
[ OK ] ex8_case1: 0x00000000
[ OK ] ex8_case4: 0x00000001

Exercicio 12 - Pipeline completo
[ OK ] ex12_case1: 0x123457789a1cdef0
[ OK ] ex12_case2: 0x000001000370f123
[ OK ] ex12_case3: 0xfffffffff7ffff
[ OK ] ex12_case4: 0x13579bdf2408ace0

Resultado final: todos os 12 exercicios passaram nos testes.
```

O arquivo completo com todas as linhas de validação foi preservado em `evidencias/qemu_test_output.txt` para servir como evidência textual da execução.

5. Conclusão

A entrega atende integralmente ao enunciado do TP2: os 12 exercícios foram implementados em Assembly ARM64, organizados em um projeto reproduzível, compilados para AArch64 e validados por execução cruzada em QEMU. O resultado final foi totalmente aprovado pela suíte automatizada, sem falhas.

Além da implementação técnica, a entrega inclui evidências textuais, instruções de reprodução, relatório em HTML/PDF e um template reutilizável para futuras entregas com a mesma identidade visual.